MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Bogdan Aramă FAF-242

Report

Laboratory work n.3
of Cryptography

Verified by:
Zaica Maia

Chisinau – 2026

1 Objective **3**

2 Short theory **3**

2.1 Polyalphabetic and digraphic substitution 3

2.2 The Playfair cipher 3

2.3 Adaptation to the Romanian alphabet 4

3 Implementation **5**

3.1 Alphabet and Input Validation 5

3.2 Building the Matrix 5

3.3 Preparing the Message 6

3.4 Encryption and Decryption 7

3.5 Reading Romanian Letters in a Windows Console 8

3.6 Program Menu 9

4 Results and Verification **10**

4.1 Check on the English Example of the Guide 10

4.2 Encryption of a Romanian Message 10

4.3 Decryption and the Manual Final Phase 12

4.4 A Message of Odd Length 12

4.5 Error Handling 14

5 Conclusions **16**

6 Source Code Repository **16**

Playfair Cipher

1 Objective

The objective of this laboratory work is to implement the Playfair cipher for messages written in Romanian (Task 3.1, assigned to odd numbers in the group register). The Romanian alphabet has 31 letters, so the classic 5×5 matrix has to be adapted. The program accepts only letters: for any other character it names the wrong characters and shows the allowed range. The key must have at least 7 letters. The user chooses encryption or decryption, enters the key and the message or the cryptogram, and receives the result. The last phase of decryption (removing the filler letters and adding the spaces) is done manually, as required by the assignment.

2 Short theory

2.1 Polyalphabetic and digraphic substitution

A monoalphabetic cipher (Laboratory work n.2) gives every plaintext letter one fixed image, so the letter frequencies of the language pass unchanged into the cryptogram and frequency analysis breaks it. A polyalphabetic cipher uses several substitution alphabets, so the same plaintext letter is encrypted differently at different places in the text and the single-letter frequencies are flattened.

The Playfair cipher reaches a similar effect in another way: it encrypts **pairs** of letters (digrams). The image of a letter depends on the other letter of its pair, so the letter A, for example, is encrypted differently in AZ and in ȘA (section 4.2). Strictly speaking Playfair is a digraphic substitution, a single fixed substitution applied to pairs, which is why the laboratory guide treats it together with the polyalphabetic ciphers. Frequency analysis of single letters does not work on it; an attacker has to count digrams, and there are far more of them: $25 \cdot 24 = 600$ possible pairs for the English 5×5 matrix and $30 \cdot 29 = 870$ for the Romanian 5×6 matrix used here. The cipher was proposed by Charles Wheatstone in 1854 and is named after Lord Lyon Playfair, who promoted it.

2.2 The Playfair cipher

The matrix. The key is written without repeated letters, followed by the remaining letters of the alphabet in their usual order, row by row. Only the arrangement of the matrix is secret. For a 30-cell matrix there are

30! ≈ 2.65 × 10^32 possible arrangements,

although fewer distinct ciphers, because shifting the rows or the columns cyclically gives the same encryption.

Preparation of the message. All characters that are not letters are removed and the text is split into pairs. If a pair would contain two equal letters, a rare filler letter is inserted between them, and if the final length is odd, a filler is added at the end. The result is a sequence of pairs of different letters.

The three rules. For each pair, the positions of its two letters in the matrix are located:

1. **Same row:** each letter is replaced by the letter to its right, the first letter of the row following the last one.
2. **Same column:** each letter is replaced by the letter below it, the top letter following the bottom one.
3. **Rectangle:** otherwise the two letters are opposite corners of a rectangle, and each letter is replaced by the letter on its own row in the column of the other letter.

Decryption applies the inverse rules: to the left on a row, upwards in a column, and the same rectangle rule. As a consequence every correct cryptogram has an even length, contains only letters of the matrix and never has a pair of two equal letters. The program uses these three properties to reject a mistyped cryptogram before decrypting it.

2.3 Adaptation to the Romanian alphabet

The Romanian alphabet has 31 letters: A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z. Since 31 is a prime number, no rectangular matrix holds exactly 31 letters. The closest one is 5 rows × 6 columns = 30 cells, so one letter has to share a cell with another. Following the classic Playfair convention, which is one of the options allowed by the laboratory guide, **J is merged into I**: J is replaced by I in the key and in the message, and the cryptogram never contains J. The cost of this choice is that a J of the original message comes back as I and has to be restored by hand in the final phase, from the context (for example IOACĂ → JOACĂ).

Filler letters. For English the guide suggests Q, X or Z as fillers, because they are rare in English. In Romanian, however, Z is a common letter (AZI, ZIUA, ZECE), so a Z inserted by the cipher could be confused with a real Z when the fillers are removed by hand. Z was therefore replaced by **W**, and the program uses the fillers **X, Q, W**, letters that appear almost only in foreign words. Consecutive fillers cycle through the list, so a triple letter such as III becomes IX IQ I…, and if the doubled letter is itself the current filler (for example XX), the next filler is taken.

Input. The message is case-insensitive and spaces are ignored. The old cedilla forms Ş and Ţ, still produced by some keyboard layouts, are converted to the correct comma-below letters Ș and Ț. Any other character (digits, punctuation, letters of other alphabets) is rejected with the list of the wrong characters and the allowed range. The key length (at least 7 letters) is counted after the spaces are removed and before repeated letters are removed.

3 Implementation

The program is written in Java (package **lab3**) and is structured into six classes:

Class	Responsibility
RomanianAlphabet.java	The 31 Romanian letters, the 30-letter matrix alphabet without J, input validation with the allowed range, normalization (uppercase, no spaces, Ş/Ţ → Ș/Ț)
PlayfairMatrix.java	Builds the matrix from the key (distinct key letters, then the rest of the alphabet), stores the position of every letter; the 5×6 Romanian matrix
PlayfairCipher.java	Preparation of the message (J → I, fillers X, Q, W), the three rules for encryption and decryption, validation of a cryptogram
ConsoleInput.java	Reads input lines; in a Windows console it reads them through PowerShell so that Ă Â Î Ș Ț typed by the user are kept
GuideExampleCheck.java	Checks the cipher on the English example of the laboratory guide (key FIRST AMENDMENT, 5×5 matrix)
Main.java	Console menu: choice of operation, reading and validating the key and the message, printing the matrix, the pairs and the result

3.1 Alphabet and Input Validation

The input is validated before it is converted to uppercase, so that a character such as the German ß cannot turn into valid letters. Every wrong character is listed once, followed by the allowed range:

```
public static void validateText(String input) {
    Set<String> invalid = new LinkedHashSet<>();

    for (char c : fixCedillas(input).replaceAll("\\s+", "").toCharArray()) {
        if (ALLOWED_CHARS.indexOf(c) < 0) {
            invalid.add(Character.isISOControl(c) ? String.format("U+%04X", (int) c) : "'" + c + "'");
        }
    }

    if (!invalid.isEmpty()) {
        throw new IllegalArgumentException("Invalid characters: " + String.join(", ", invalid)
                + ". Allowed range: " + ALLOWED_RANGE + ". Spaces are ignored.");
    }
}
```

3.2 Building the Matrix

A **LinkedHashSet** keeps the letters in the order of their first appearance and ignores repetitions, so adding the key and then the whole alphabet gives exactly the order of the matrix. The position of every letter is stored in a map, so that a pair is located without searching the matrix:

```
public PlayfairMatrix(String key, String alphabet, int rows, int cols) {
    if (rows * cols != alphabet.length()) {
        throw new IllegalArgumentException("A " + rows + "x" + cols + " matrix cannot hold "
                + alphabet.length() + " letters.");
    }

    Set<Character> letters = new LinkedHashSet<>();
    for (char c : key.toCharArray()) {
        if (alphabet.indexOf(c) < 0) {
            throw new IllegalArgumentException("Key letter '" + c + "' is not in the matrix alphabet.");
        }
        letters.add(c);
    }

    StringBuilder sb = new StringBuilder();
    for (char c : letters) {
        sb.append(c);
    }
    distinctKey = sb.toString();

    for (char c : alphabet.toCharArray()) {
        letters.add(c);
    }

    cells = new char[rows][cols];
    int i = 0;
    for (char c : letters) {
        cells[i / cols][i % cols] = c;
        positions.put(c, new int[]{i / cols, i % cols});
        i++;
    }
}
```

```
public static PlayfairMatrix forRomanian(String key) {
    return new PlayfairMatrix(key.replace('J', 'I'), RomanianAlphabet.MATRIX_ALPHABET,
            ROMANIAN_ROWS, ROMANIAN_COLS);
}
```

3.3 Preparing the Message

The message is prepared in a single pass. When the two letters of a pair are equal, only the first one is used and the second starts the next pair, exactly as in the hand method. The same branch completes a final single letter:

```java
public static String prepare(String text) {
    String letters = text.replace('J', 'I');
    StringBuilder prepared = new StringBuilder();
    int filler = 0;
    int i = 0;

    while (i < letters.length()) {
        char first = letters.charAt(i);

        if (i + 1 < letters.length() && letters.charAt(i + 1) != first) {
            prepared.append(first).append(letters.charAt(i + 1));
            filler = 0;
            i += 2;
        } else {
            // Equal letters: only the first one is used now, the other starts the next pair
            if (FILLERS[filler % FILLERS.length] == first) {
                filler++;
            }
            prepared.append(first).append(FILLERS[filler % FILLERS.length]);
            filler++;
            i++;
        }
    }

    return prepared.toString();
}
```

3.4 Encryption and Decryption

Encryption and decryption differ only in the direction of the shift: +1 for encryption and −1 for decryption. Adding the number of columns (or rows) before the remainder keeps the index positive when it moves to the left of the first column. The rectangle rule is its own inverse:

```
private static String transform(String text, PlayfairMatrix matrix, int shift) {
    StringBuilder result = new StringBuilder();
    int rows = matrix.rows();
    int cols = matrix.cols();

    for (int i = 0; i < text.length(); i += 2) {
        int[] a = matrix.positionOf(text.charAt(i));
        int[] b = matrix.positionOf(text.charAt(i + 1));

        if (a[0] == b[0]) {
            // Same row: next (or previous) letter on the row, wrapping around
            result.append(matrix.at(a[0], (a[1] + shift + cols) % cols));
            result.append(matrix.at(b[0], (b[1] + shift + cols) % cols));
        } else if (a[1] == b[1]) {
            // Same column: letter below (or above), wrapping around
            result.append(matrix.at((a[0] + shift + rows) % rows, a[1]));
            result.append(matrix.at((b[0] + shift + rows) % rows, b[1]));
        } else {
            // Rectangle: same row, column of the other letter
            result.append(matrix.at(a[0], b[1]));
            result.append(matrix.at(b[0], a[1]));
        }
    }

    return result.toString();
}
```

Before decryption, the cryptogram is checked against the properties from section 2.2:

```
public static void validateCiphertext(String text) {
    if (text.indexOf('J') >= 0) {
        throw new IllegalArgumentException("A cryptogram cannot contain J, because J is not in the matrix.");
    }
    if (text.length() % 2 != 0) {
        throw new IllegalArgumentException("A cryptogram must have an even number of letters. Provided: "
                + text.length() + ".");
    }
    for (int i = 0; i < text.length(); i += 2) {
        if (text.charAt(i) == text.charAt(i + 1)) {
            throw new IllegalArgumentException("Pair " + (i / 2 + 1) + " has two equal letters ("
                    + text.substring(i, i + 2) + "), which encryption never produces.");
        }
    }
}
```

3.5 Reading Romanian Letters in a Windows Console

While testing on Windows it turned out that Java 17 reads Romanian letters typed in the console as NUL or '?', whatever the code page of the console is; this was checked with the code pages 65001 (UTF-8) and 852. The class **ConsoleInput** therefore asks PowerShell to read the line through the Unicode console API and to return it encoded in Base64, which contains only ASCII characters and survives the transfer to Java. When the input comes from a pipe or a file, or PowerShell is not available, an ordinary UTF-8 Scanner is used:

```
private static String readWithPowerShell() {
    try {
        Process process = new ProcessBuilder("powershell", "-NoProfile", "-Command", READ_LINE)
                .redirectInput(ProcessBuilder.Redirect.INHERIT)
                .redirectError(ProcessBuilder.Redirect.DISCARD)
                .start();
        String encoded = new String(process.getInputStream().readAllBytes(), StandardCharsets.US_ASCII).trim();

        if (process.waitFor() != 0) return null;
        return new String(Base64.getDecoder().decode(encoded), StandardCharsets.UTF_8);
    } catch (IOException | IllegalArgumentException e) {
        return null;
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return null;
    }
}
```

Typing the diacritics in a real Windows console with this class was tested on the author's computer, at the code pages 852 and 65001. The transcripts in section 4 were produced on Linux with the input given through a pipe, where the Scanner branch is used, so they do not test the PowerShell branch.

3.6 Program Menu

The user chooses the operation from a menu. The key is requested again until it is valid; the message and the cryptogram are handled in the same way:

```
Select Operation:
1. Encrypt
2. Decrypt
0. Exit
Choice: 1
```

```
private static String readKey() {
    while (true) {
        out.print("Enter key (>= " + MIN_KEY_LENGTH + " letters): ");
        String input = ConsoleInput.readLine();

        try {
            RomanianAlphabet.validateText(input);
        } catch (IllegalArgumentException e) {
            out.println("[Error] " + e.getMessage());
            continue;
        }

        String normalized = RomanianAlphabet.normalizeText(input);
        if (normalized.length() >= MIN_KEY_LENGTH) {
            return normalized;
        }
        out.println("[Error] The key must have at least " + MIN_KEY_LENGTH
                + " letters. Provided length: " + normalized.length());
    }
}
```

4 Results and Verification

All outputs in this section are real outputs of the program, compiled with `javac -encoding UTF-8 lab3/*.java` and run from the root of the repository. Because input given through a pipe is not echoed, the inputs were sent by a small script that also writes each input into the transcript right after the prompt that read it. The results were also compared with a separate implementation of the three rules written for the check; they agree for every example below.

4.1 Check on the English Example of the Guide

The laboratory guide encrypts the First Amendment of the U.S. Constitution with the key FIRST AMENDMENT and a 5×5 matrix without J. **GuideExampleCheck** builds this matrix with the same **PlayfairMatrix** class, encrypts the prepared text of the guide and decrypts the cryptogram of the guide:

```
$ javac -encoding UTF-8 lab3/*.java
$ java lab3.GuideExampleCheck
Matrix:
F I R S T
A M E N D
B C G H K
L O P Q U
V W X Y Z
Pairs in the guide : 115
Encryption matches : true
Decryption matches : true
```

All 115 pairs agree in both directions. The prepared text printed in the guide contains a few slips of its own: „shall make” is prepared as AL **LM MA** KE, which repeats the M; „abridging” is prepared as A**BR RI**DGING, which adds an R; and in „prohibiting the free” the letters HE are lost (IN **GT FR**). For this reason the check uses the guide's prepared text exactly as it is printed and does not prepare the English sentence again; otherwise it would not be comparable with the guide's cryptogram.

4.2 Encryption of a Romanian Message

The message „AZI FIII LUI ION JOACĂ ȘAH ÎN GRĂDINĂ” (today the son of Ion plays chess in the garden) was chosen because it contains a triple letter (FIII), a double letter across two words (LUI ION), the letter J, the letter Z and the diacritics Ă, Î and Ș. The key is CRIPTOGRAFIE:

```
--- Playfair Encryption ---
Enter key (>= 7 letters): CRIPTOGRAFIE

[Key without repeated letters]: CRIPTOGAFE
[Encryption Matrix 5x6]:
C R I P T O
G A F E Ă Â
B D H Î K L
M N Q S Ș Ț
U V W X Y Z

Enter plaintext: AZI FIII LUI ION JOACĂ ȘAH ÎN GRĂDINĂ

[Result] Normalized Plaintext : AZIFIIILUIIONJOACĂȘAHÎNGRĂDINĂ
[Result] Prepared Pairs       : AZ IF IX IQ IL UI IO NI OA CĂ ȘA HÎ NG RĂ DI NĂ
[Result] Ciphertext Pairs     : ÂV FH PW FW OH WC PC QR RÂ TG NĂ ÎK MA TA HR ȘA
[Result] Ciphertext           : ÂVFHPWFWOHWCPCQRRÂTGNĂÎKMATAHRȘA
```

	1	2	3	4	5	6
1	**C**	**R**	**I**	**P**	**T**	**O**
2	**G**	**A**	**F**	**E**	**Ă**	**Â**
3	**B**	**D**	**H**	**Î**	**K**	**L**
4	**M**	**N**	**Q**	**S**	**Ș**	**Ț**
5	**U**	**V**	**W**	**X**	**Y**	**Z**

Figure 4.1. Matrix for the key CRIPTOGRAFIE (the distinct key letters are shaded)

The message has 30 letters. During preparation, J in JOACĂ becomes I (pair NI), and the triple I of FIII receives two different fillers: FI is a normal pair, then the next two I's give IX and IQ, and the last I forms IL with the L of LUI. The double I of LUI ION falls into two different pairs (UI IO), so it needs no filler. The prepared text has 32 letters, an even number, so no filler is added at the end. All three rules occur in this example:

#	Pair	Positions (row, col)	Rule	Result
1	**AZ**	A(2,2) Z(5,6)	rectangle	**ÂV**
2	**IF**	I(1,3) F(2,3)	same column	**FH**
3	**IX**	I(1,3) X(5,4)	rectangle	**PW**
4	**IQ**	I(1,3) Q(4,3)	same column	**FW**
5	**IL**	I(1,3) L(3,6)	rectangle	**OH**
6	**UI**	U(5,1) I(1,3)	rectangle	**WC**
7	**IO**	I(1,3) O(1,6)	same row	**PC**
8	**NI**	N(4,2) I(1,3)	rectangle	**QR**

#	Pair	Positions (row, col)	Rule	Result
9	**OA**	O(1,6) A(2,2)	rectangle	**RÂ**
10	**CĂ**	C(1,1) Ă(2,5)	rectangle	**TG**
11	**ȘA**	Ș(4,5) A(2,2)	rectangle	**NĂ**
12	**HÎ**	H(3,3) Î(3,4)	same row	**ÎK**
13	**NG**	N(4,2) G(2,1)	rectangle	**MA**
14	**RĂ**	R(1,2) Ă(2,5)	rectangle	**TA**
15	**DI**	D(3,2) I(1,3)	rectangle	**HR**
16	**NĂ**	N(4,2) Ă(2,5)	rectangle	**ȘA**

Table 4.1. Encryption of the pairs with the matrix of figure 4.1 (row and column numbered from 1)

For example, I and F are in column 3, so each is replaced by the letter below it: FH. H and Î are on row 3, so each moves one place to the right: HÎ → ÎK. A and Z form a rectangle, so A takes the letter of its own row in the column of Z (Â) and Z takes the letter of its own row in the column of A (V): ÂV.

4.3 Decryption and the Manual Final Phase

The cryptogram is decrypted with the same key:

```
Choice: 2

--- Playfair Decryption ---
Enter key (>= 7 letters): CRIPTOGRAFIE

[Key without repeated letters]: CRIPTOGAFE
[Encryption Matrix 5x6]:
C R I P T O
G A F E Ă Â
B D H Î K L
M N Q S Ș Ț
U V W X Y Z

Enter ciphertext: ÂVFHPWFWOHWCPCQRRÂTGNĂÎKMATAHRȘA

[Result] Normalized Ciphertext: ÂVFHPWFWOHWCPCQRRÂTGNĂÎKMATAHRȘA
[Result] Decrypted Pairs      : AZ IF IX IQ IL UI IO NI OA CĂ ȘA HÎ NG RĂ DI NĂ
[Result] Decrypted Plaintext  : AZIFIXIQILUIIONIOACĂȘAHÎNGRĂDINĂ
[Info] Remove the filler letters, restore J where needed and add the spaces manually.
```

The program stops at the decrypted text, as required. The final phase is done by hand:

Step	Text
Decrypted text	AZIFIXIQILUIIONIOACĂȘAHÎNGRĂDINĂ
Remove the fillers X and Q between the I's	AZIFIIILUIIONIOACĂȘAHÎNGRĂDINĂ
Restore J (IOACĂ is not a word, JOACĂ is)	AZIFIIILUIIONJOACĂȘAHÎNGRĂDINĂ
Add the spaces	AZI FIII LUI ION JOACĂ ȘAH ÎN GRĂDINĂ

Table 4.2. Manual final phase for the first message

This step needs the reader's judgement: the program cannot know whether an X between two equal letters is a filler, and whether an I was a J. Because X, Q and W are rare in Romanian, the fillers are easy to recognize, whereas a Z filler could have been confused with the Z of AZI.

4.4 A Message of Odd Length

The second message, „ÎNTÂLNIREA SECRETĂ ARE LOC MÂINE DIMINEAȚĂ” (the secret meeting takes place tomorrow morning), has 37 letters and no pair of equal letters, so the only filler is the one that completes the last pair. The key is UNIVERSITATE:

```
Choice: 1

--- Playfair Encryption ---
Enter key (>= 7 letters): UNIVERSITATE

[Key without repeated letters]: UNIVERSTA
[Encryption Matrix 5x6]:
U N I V E R
S T A Ă Â B
C D F G H Î
K L M O P Q
Ș Ț W X Y Z

Enter plaintext: ÎNTÂLNIREA SECRETĂ ARE LOC MÂINE DIMINEAȚĂ

[Result] Normalized Plaintext : ÎNTÂLNIREASECRETĂARELOCMÂINEDIMINEAȚĂ
[Result] Prepared Pairs       : ÎN TÂ LN IR EA SE CR ET ĂA RE LO CM ÂI NE DI MI NE AȚ ĂX
[Result] Ciphertext Pairs     : DR AB ȚT VU IÂ ÂU ÎU NÂ ÂĂ UR MP FK AE IR FN WA IR TW GV
[Result] Ciphertext           : DRABȚTVUIÂÂUÎUNÂÂĂURMPFKAEIRFNWAIRTWGV
```

	1	2	3	4	5	6
1	**U**	**N**	**I**	**V**	**E**	**R**
2	**S**	**T**	**A**	**Ă**	**Â**	**B**
3	**C**	**D**	**F**	**G**	**H**	**Î**
4	**K**	**L**	**M**	**O**	**P**	**Q**
5	**Ș**	**Ț**	**W**	**X**	**Y**	**Z**

Figure 4.2. Matrix for the key UNIVERSITATE (the distinct key letters are shaded)

The repeated letters of the key are dropped (UNIVERSITATE → UNIVERSTA). The last letter Ă is completed with the filler X, so the prepared text has 38 letters (19 pairs). The last pair ĂX also shows the wrap-around: Ă and X are in column 4, and X, on the last row, is replaced by V from the first row. The cryptogram was entered for decryption in groups of four letters, to show that spaces are ignored:

```
Choice: 2

--- Playfair Decryption ---
Enter key (>= 7 letters): UNIVERSITATE

[Key without repeated letters]: UNIVERSTA
[Encryption Matrix 5x6]:
U N I V E R
S T A Ă Â B
C D F G H Î
K L M O P Q
Ș Ț W X Y Z

Enter ciphertext: DRAB ȚTVU IÂÂU ÎUNÂ ÂĂUR MPFK AEIR FNWA IRTW GV

[Result] Normalized Ciphertext: DRABȚTVUIÂÂUÎUNÂÂĂURMPFKAEIRFNWAIRTWGV
[Result] Decrypted Pairs      : ÎN TÂ LN IR EA SE CR ET ĂA RE LO CM ÂI NE DI MI NE AȚ ĂX
[Result] Decrypted Plaintext  : ÎNTÂLNIREASECRETĂARELOCMÂINEDIMINEAȚĂX
[Info] Remove the filler letters, restore J where needed and add the spaces manually.
```

In the final phase only the X at the end is removed, and the spaces are added: ÎNTÂLNIREA SECRETĂ ARE LOC MÂINE DIMINEAȚĂ.

4.5 Error Handling

An invalid menu option is reported and the menu is shown again:

```
Choice: 3
[Error] Invalid option. Please enter 0, 1, or 2.
```

A key that is too short or contains digits is requested again. A message with digits and punctuation is rejected with the list of the wrong characters and the allowed range; the message written in words and in lowercase is then accepted and normalized:

```
Choice: 1

--- Playfair Encryption ---
Enter key (>= 7 letters): CHEIE
[Error] The key must have at least 7 letters. Provided length: 5
Enter key (>= 7 letters): CHEIA 2026
[Error] Invalid characters: '2', '0', '6'. Allowed range: 'A'-'Z', 'a'-'z' and the Romanian
letters Ă Â Î Ș Ț / ă â î ș ț. Spaces are ignored.
Enter key (>= 7 letters): CHEIA SECRETĂ

[Key without repeated letters]: CHEIASRTĂ
[Encryption Matrix 5x6]:
C H E I A S
R T Ă Â B D
F G Î K L M
N O P Q Ș Ț
U V W X Y Z

Enter plaintext: Întâlnire la ora 10:30, în sala 3-12!
[Error] Invalid characters: '1', '0', ':', '3', ',', '-', '2', '!'. Allowed range: 'A'-'Z',
'a'-'z' and the Romanian letters Ă Â Î Ș Ț / ă â î ș ț. Spaces are ignored.
Enter plaintext: întâlnire la ora zece în sala trei

[Result] Normalized Plaintext : ÎNTÂLNIRELAORAZECEÎNSALATREI
[Result] Prepared Pairs       : ÎN TÂ LN IR EL AO RA ZE CE ÎN SA LA TR EI
[Result] Ciphertext Pairs     : FP ĂB FȘ CÂ AÎ HȘ BC WS HI FP CS ȘB ĂT IA
[Result] Ciphertext           : FPĂBFȘCÂAÎHȘBCWSHIFPCSȘBĂTIA
```

A cryptogram is rejected if it contains J, has an odd length or contains a pair of two equal letters. The last input, in lowercase and with a space, is accepted (it is the beginning of the cryptogram of section 4.2):

```
Choice: 2

--- Playfair Decryption ---
Enter key (>= 7 letters): CRIPTOGRAFIE

[Key without repeated letters]: CRIPTOGAFE
[Encryption Matrix 5x6]:
C R I P T O
G A F E Ă Â
B D H Î K L
M N Q S Ș Ț
U V W X Y Z

Enter ciphertext: ÂVFH JWFW
[Error] A cryptogram cannot contain J, because J is not in the matrix.
Enter ciphertext: ÂVFHPWFWO
[Error] A cryptogram must have an even number of letters. Provided: 9.
Enter ciphertext: ÂVFHPPFWOH
[Error] Pair 3 has two equal letters (PP), which encryption never produces.
Enter ciphertext: âvfh pwfw

[Result] Normalized Ciphertext: ÂVFHPWFW
[Result] Decrypted Pairs      : AZ IF IX IQ
[Result] Decrypted Plaintext  : AZIFIXIQ
[Info] Remove the filler letters, restore J where needed and add the spaces manually.
```

5 Conclusions

In this laboratory work the Playfair cipher was implemented in Java for the 31 letters of the Romanian alphabet. The key findings are:

1. **The alphabet must fit the matrix:** 31 is prime, so no rectangle holds all the Romanian letters. A 5×6 matrix with J merged into I keeps all the diacritics in the matrix; the price is that J has to be restored by hand after decryption.
2. **Fillers must be rare in the language of the message:** the English fillers Q, X, Z do not suit Romanian, where Z is common. With X, Q and W, the fillers in the decrypted texts of section 4 were easy to recognize and remove.
3. **Encrypting pairs hides single-letter frequencies:** the same plaintext letter has different images depending on its partner (A → Â in AZ and OA, but A → Ă in ȘA), so the frequency analysis of Laboratory work n.2 cannot be applied directly. The cipher is still a fixed substitution of pairs and leaks digram statistics.
4. **The structure of a cryptogram can be checked:** an even length, no J and no pair of equal letters are necessary properties, and checking them catches many typing mistakes before decryption.
5. **The implementation was checked on the guide's example:** the program reproduces all 115 pairs of the English example in both directions, and the Romanian results agree with a separate implementation of the rules.

6 Source Code Repository

The complete source code is available in the GitHub repository:
https://github.com/bogdan-arama/cs-labs/tree/main/lab3